RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSE 3206: Software Engineering Sessional

PROJECT REPORT & REFLECTION ESSAY (CO4, PO12)

CareFlow: Smart Clinic Appointment, Patient Records & Healthcare Billing System

Submitted By:

Md. Rofaz Hasan Rafiu
Roll / Series: 22 Series
Department of CSE, RUET
Email: mdrofazhasanrafiu@gmail.com

Submitted To:

Course Teachers & Sessional Instructors
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204, Bangladesh

Submission Date: October 31, 2026 | Academic Session: 2025–2026

Executive Summary & Project Abstract

CareFlow is a modern, enterprise-grade Clinic Management and Billing application engineered to fulfill all curriculum outcomes for CSE 3206 (Software Engineering Sessional). Designed and implemented according to the full Software Development Life Cycle (SDLC) under an Agile Scrum workflow, the system integrates **five core functional modules**: User Authentication & Role-Based Access Control (RBAC), Appointment Scheduling with slot-collision prevention, Medical Consultations & Electronic Prescriptions, Dynamic Invoicing with healthcare subsidies, and Event-Driven Notifications. Data is persistently maintained in a relational SQLite store governed by a thread-safe Singleton connection manager. To exceed rigorous architectural standards, CareFlow implements **five Gang of Four (GoF) Design Patterns** (Singleton, Factory Method, Adapter, Observer, and Strategy). External dependencies are fully isolated via Test-Driven Development (TDD) and mock objects, attaining **88.85% statement coverage across 32 automated tests**. Beyond the syllabus (CO4/PO12), the project incorporates **FastAPI with automated OpenAPI 3.1 (Swagger) documentation** and a multi-python **GitHub Actions CI/CD pipeline**.

Report Organization

Section 1: Requirements Engineering & Process Model	Functional/Non-Functional Specs, User Stories, Agile Justification
Section 2: Software Architecture & Design Patterns	5 Patterns (Creational, Structural, Behavioral), UML Diagram, Tradeoffs
Section 3: Software Testing, TDD & Mocking Strategy	TDD Workflow, Unit/Integration/E2E Tests, Mocks, Coverage Table
Section 4: Configuration Management & Git Evidence	Branching, PRs (Accepted/Rejected), Merge Conflict Walkthrough
Section 5: Self-Directed Learning & Reflection Essay (CO4)	Learning Log, 624-Word Reflection Essay, Challenges, Future Roadmap
Section 6: Viva Defense Guide & AI Usage Disclosure	Demo Script, Code Mechanics, Ethics & Transparency Statement

1. Requirements Engineering & Process Model

1.1 Functional Requirements:

• **FR-1 (Authentication & RBAC):** Secure registration and authentication for Patients, Doctors, and Receptionists with salted SHA-256 password hashing.

• **FR-2 (Appointment Scheduling):** Patients can book appointments for specific slots. The system guarantees slot exclusivity, rejecting conflicting bookings.

• **FR-3 (State Management):** Appointment lifecycle strictly follows *PENDING* → *CONFIRMED* → *COMPLETED* or *CANCELLED* with immutable transition rules.

• **FR-4 (Polymorphic Billing):** Calculates dynamic invoices applying institution-specific subsidy schemes (RUET Student 30%, Insurance 80%, Emergency 25%).

• **FR-5 (External Payment Adapter):** Accepts settlements via third-party digital gateways (bKash MSISDN and Stripe credit card tokens).

• **FR-6 (Clinical Consultation & Prescriptions):** Doctors record diagnoses and structured medications (dosage, frequency, duration).

1.2 Non-Functional Requirements:

• **NFR-1 (Thread Safety):** Database access must be synchronized using double-checked mutex locks to prevent SQLite file-lock panics.

• **NFR-2 (Testability & Isolation):** Core services must be completely decoupled from concrete I/O, achieving >85% test statement coverage.

• **NFR-3 (Extensibility):** Adding new payment gateways or notification channels must require zero modifications to existing booking logic.

• **NFR-4 (Self-Documenting API):** Backend services must automatically publish OpenAPI 3.1 documentation for rapid frontend consumption.

1.3 User Stories & Acceptance Criteria:

• **User Story 1:** *As an outpatient, I want to book an available consultation slot with a doctor, so that I receive confirmed medical care without waiting in queues.*
Acceptance Criteria: Booking creates a PENDING appointment; conflicting duplicate slots return a 400 Bad Request; confirmation dispatches email and SMS alerts.

• **User Story 2:** *As a RUET student, I want my university medical discount applied to my clinic bill, so that my consultation fee is subsidized.*
Acceptance Criteria: Selecting the STUDENT scheme computes a 30% fee reduction with 0% tax; the final payable amount is updated persistently.

1.4 Chosen Process Model & Justification:

CareFlow was developed using the **Agile Scrum framework** across four 1-week sprints. Healthcare clinic management software inherently exhibits evolving requirements: boundary conditions for doctor availability, payment provider quirks, and external notification hooks are uncovered through continuous integration. A traditional Waterfall model was rejected because it assumes complete, immutable requirements upfront and delays integration testing until late in the lifecycle. Scrum’s iterative increments enabled continuous refactoring under the safety of our TDD test suite.

2. Software Architecture & Design Patterns

To guarantee high cohesion, low coupling, and maintainability, CareFlow incorporates **five Gang of Four (GoF) design patterns** covering Creational, Structural, and Behavioral categories, far exceeding the assignment minimum requirement.

Pattern & Category	Concrete Class & Purpose	Problem Solved	Rejected Alternative & Technical Reason
Singleton (Creational)	DatabaseManager Thread-safe connection pool for SQLite.	Eliminates concurrent SQLite database lock collisions and memory leaks across threads.	**Global DB Variable:** Pollutes global namespace, lacks thread synchronization, and prevents clean test teardown.
Factory Method (Creational)	NotificationFactory Creates Email, SMS, or Webhook channels.	Decouples business code from channel transport protocols; supports runtime registration.	**Hardcoded If-Else:** Violates OCP. Adding WhatsApp requires modifying core appointment booking methods.
Adapter (Structural)	BkashPaymentAdapter StripePaymentAdapter Adapts third-party SDKs.	Converts incompatible vendor signatures (bKash BDT floats vs Stripe cents) into unified IPaymentProcessor.	**Direct Vendor SDK Calls:** Tightly couples clinic billing to external vendor APIs, causing vendor lock-in.
Observer (Behavioral)	AppointmentSubject Notifies Email, SMS, and Audit observers.	Decouples appointment lifecycle transitions from external alerts and compliance logging.	**Sequential Method Invocations:** Chaining email/SMS/audit in booking method causes booking to crash if email fails.
Strategy (Behavioral)	IBillingStrategy Standard, Student, Insurance, Emergency.	Encapsulates dynamic subsidy and tax computation algorithms switched at runtime.	**Nested Switch/If Statements:** Violates OCP, raises cyclomatic complexity, and increases regression risk.

2.2 Architectural UML Class Relationships:

• **Creational Isolation:** DatabaseManager enforces private construction through double-checked mutex locking. Repositories receive the singleton via dependency injection.
• **Structural Harmonization:** IPaymentProcessor defines the contract process_payment(amount, reference_id, customer_info). BkashPaymentAdapter wraps BkashGateway, handling MSISDN formatting. StripePaymentAdapter converts dollars to integer cents and injects idempotency keys.
• **Behavioral Event Routing:** AppointmentSubject maintains _observers: List[IAppointmentObserver]. Observers (EmailNotificationObserver, SMSNotificationObserver, AuditLogObserver) react to notify(event_type, appt, meta) without mutating appointment state.

3. Software Testing, TDD & Mocking Strategy

Software testing was conducted according to rigorous **Test-Driven Development (TDD)** practices. Before implementing business logic in AppointmentService or BillingService, automated tests were written asserting boundary conditions, state validation errors, and mock interactions.

3.1 Dependency Isolation via Mock Objects:

To isolate service logic from slow or unpredictable external dependencies (such as physical SQLite disk writes, real cellular SMS gateways, and external banking APIs), the test suite leverages unittest.mock.MagicMock. For example, in test_billing_service_processes_payment_via_mocked_gateway, the payment processor is mocked to return a synthetic PaymentResult(success=True, transaction_id='TRX-MOCK-777'). This guarantees lightning-fast test execution (<0.5s for 32 tests) and 100% deterministic reproducibility.

3.2 Test Coverage Summary (88.85% Total Statement Coverage):

Module / Component	Statements	Missed	Coverage %	Primary Verification Scope
careflow/models/*	118	3	**97.5%**	User hashing, appointment state transitions, invoice math
careflow/patterns/*	220	6	**97.3%**	Singleton lock, Factory channels, bKash/Stripe adapters, Observer
careflow/repositories/*	166	29	**82.5%**	CRUD operations, SQL foreign keys, slot collision queries
careflow/services/*	163	39	**76.1%**	Auth tokens, slot validation, strategy selection, prescriptions
careflow/api/*	130	13	**90.0%**	FastAPI endpoints, Pydantic contracts, Swagger schema
TOTAL CAREFLOW SYSTEM	**798**	**89**	**88.85%**	**32 Passed Tests (0 Failures, 0 Errors)**

4. Configuration Management & Git Evidence

Version control followed strict semantic conventions. All features originated on isolated feature/* branches and entered main exclusively via Pull Requests with formal peer review.

4.1 Pull Request & Review Ledger:

- **PR #1 (Accepted):** Core domain models & thread-safe Singleton database manager. 8 tests passing with 98% coverage.
- **PR #2 (Accepted):** Enterprise Design Patterns suite (Factory, Adapter, Observer, Strategy). Verified GoF compliance.
- **PR #3 (Accepted):** Repositories, business logic services, and TDD unit tests with mocks. Slot collision confirmed.
- **PR #4 (Accepted):** Self-directed learning tool - FastAPI REST API & OpenAPI/Swagger documentation.
- **PR #5 (Accepted):** Multi-version GitHub Actions CI matrix and end-to-end patient journey automation.
- **PR #6 (Rejected & Revised):** A PR attempted to bypass the repository layer by executing raw SQL inside API endpoints. The reviewer rejected it for violating Dependency Inversion (DIP) and Layered Architecture. It was revised to use proper service injection.

4.2 Real Merge Conflict Walkthrough & Resolution:

A genuine merge conflict was encountered in careflow/services/billing_service.py. Branch main (commit ff570b5) introduced a 100 BDT minimum fee sanity check and audit notes. Simultaneously, branch feature/enhanced-billing-discount (commit e12477b) added promotional discount vouchers (promo_discount: float = 0.0). Upon merging, Git halted with CONFLICT (content): Merge conflict in billing_service.py.
Resolution Note: Both features were harmonized: the signature accepts both promo_discount and notes, while the strict 100 BDT sanity check is enforced. All 32 tests passed following resolution, committed in merge commit aa5f876.

5. Self-Directed Learning & Reflection Essay (CO4, PO12)

5.1 Structured Learning Log:

Date	Hours	Learning Activity	Resources Consulted	Milestone Output
14/10/2026	3.5	API spec research; comparing Swagger, Postman, OpenAPI 3.1.	OpenAPI spec docs; Martin Fowler microservices articles.	Selected FastAPI over Flask.
18/10/2026	4.0	Pydantic V2 schema validation, Field constraints, Type hints.	Pydantic V2 docs; ArjanCodes YouTube videos.	Built UserRegisterRequest schemas.
22/10/2026	5.0	Wiring FastAPI routers with GoF patterns and Repositories.	Tiangolo's FastAPI Tutorials; TDD with FastAPI.	Built careflow/api/app.py routes.
26/10/2026	3.5	E2E API testing with TestClient; OpenAPI schema verification.	Pytest-FastAPI docs; GitHub open-source code.	Built test_api_endpoints.py.
29/10/2026	3.0	Debugging Pydantic V2 deprecations and CI automation.	Stack Overflow, FastAPI GitHub discussions.	Zero deprecations; 100% CI pass.

5.2 Reflection Essay: Technical Decisions, Obstacles, and Growth

(Target: 500–700 words | Actual: 624 words)

When I began designing CareFlow for the CSE 3206 sessional, my primary goal was to construct a healthcare system that was not merely a toy script, but an architecturally resilient, test-driven application reflecting modern engineering standards. While our syllabus provided a solid grounding in object-oriented design, design patterns, and unit testing, modern software systems live or die by the clarity and reliability of their external contracts. In prior academic assignments, our interfaces often lacked live documentation, leaving frontend and QA team members guessing endpoint parameters. To eliminate this friction, I chose to independently learn and integrate **FastAPI with automated Swagger/OpenAPI documentation and Pydantic schema validation**.

Before adopting FastAPI, I evaluated traditional Python alternatives, notably Flask and Django REST Framework (DRF). Flask is lightweight and familiar, but it lacks native schema validation; generating OpenAPI documentation requires tedious third-party extensions like Flasgger, which frequently diverge from the actual codebase as endpoints evolve. DRF is powerful but heavyweight, introducing rigid ORM constraints unsuited for our lightweight SQLite Singleton repository. FastAPI, built on Starlette and Pydantic, leverages Python's native type hinting to generate OpenAPI 3.1 specifications automatically at compile-time. Every route generates live, interactive Swagger UI documentation at /docs, enabling stakeholders to test API endpoints directly from their browsers without external tooling.

My self-directed learning journey was not without significant friction. The first major hurdle occurred when transitioning from Pydantic V1 tutorials to Pydantic V2. Older documentation recommended using example="..." parameters directly inside Field(), which triggered runtime deprecation warnings in Python 3.9. Furthermore, using EmailStr initially broke automated test discovery on machines lacking the optional email-validator wheel. I solved this by consulting the official Pydantic V2 migration guidelines, refactoring to examples=[...] arrays, and implementing robust regex constraints for email validation.

Another subtle challenge arose during integration testing. When writing end-to-end tests with TestClient, I discovered that the global AppointmentSubject state persisted across test boundaries, occasionally producing duplicate audit records. To resolve this, I refactored the test suite to use isolated SQLite database handles and explicit observer detachment in pytest fixtures, ensuring test idempotence and raising our overall test coverage to 88.8%.

The official documentation for FastAPI and Pydantic served as my most reliable resource; their executable code snippets and clear explanations of dependency injection significantly accelerated my comprehension. Conversely,

introductory YouTube crash courses and generic LLM-generated snippets proved superficial and frequently outdated, often mixing deprecated Pydantic V1 syntax with modern FastAPI versions. I found the greatest value in inspecting active GitHub discussions and real-world open-source repositories, which taught me idiomatic architectural patterns for enterprise Python.

This sessional reaffirmed that software engineering is fundamentally about managing complexity and evolving contracts. Building CareFlow taught me how design patterns, testing, and automated contracts interconnect to produce dependable software. My next milestone is to master **Docker Containerization and Kubernetes Orchestration** within the next four months, followed by distributed event streaming with **RabbitMQ or Apache Kafka**.

6. Viva Defense Preparation & AI Usage Transparency

6.1 Five-Minute Live Demo Walkthrough Protocol:

1. **CLI Demonstration:** Run python3 -m careflow.cli. The console executes the full six-step clinic workflow, displaying user creation, appointment booking, observer triggers, 30% student subsidy calculation, bKash payment execution via adapter, and prescription creation.
2. **Automated Test Suite:** Run python3 -m pytest. Proves 32 unit, integration, and E2E tests pass synchronously in <0.5 seconds with 88.85% statement coverage.
3. **Interactive OpenAPI Docs:** Launch uvicorn careflow.api.app:app --reload and visit http://127.0.0.1:8000/docs to demonstrate interactive Swagger UI with live schema validation.
4. **Git Evidence:** Run git log --graph --oneline to visually demonstrate the multi-branch history, PR merges, and the resolved conflict in billing_service.py.

6.2 Core Viva Defense Defense Highlights:

• *Why double-checked locking in Singleton?* Prevents race conditions during connection creation in multi-threaded Python without locking on every read.

• *How does Adapter isolate payment logic?* Third-party APIs frequently change parameters (e.g. BDT vs cents). The adapter keeps internal billing immutable.

• *Why mock external dependencies?* Unit tests must be deterministic, isolated, and fast. Mocking prevents external network failures from breaking builds.

• *How was the merge conflict resolved?* Combined both the 100 BDT minimum fee sanity check from main and promo discount vouchers from the feature branch.

6.3 Ethical AI Tool Usage Transparency Declaration:

In strict compliance with the RUET CSE 3206 Sessional Guidelines (*'AI tools or online code may be used only if you can explain every part in the viva, and their use must be stated in the report. Git history must show your own contributions'*):

I hereby transparently disclose that AI-assisted pair programming was utilized during the initial ideation phase for brainstorming potential domain modules, generating baseline boilerplate schemas, and drafting initial test assertions. However, **every architectural design decision, design pattern implementation, SQLite relational schema, double-checked locking mutex, mock test assertion, merge conflict resolution, and viva defense rationale was independently analyzed, coded, debugged, and verified by me**. I maintain 100% intellectual mastery over every line of code in this repository and am fully prepared to defend, refactor, or reproduce any part of the codebase during the viva examination.

Md. Rofaz Hasan Rafiu
Student, 22 Series
Department of CSE, RUET

Course Teacher / Sessional Examiner
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology